

Conditional RMSD for Rasch item fit, Part A

Null behaviour, stability in n, leakage, discrimination and targeting, without a bootstrap

Magnus Johansson

2026-10-05

Table of contents

Purpose	1
Definitions	1
Predictions, stated before running	2
Design	2
Data generation	4
Statistics	5
Calibrating the LPE item	6
Run	9
Checks	10
Q1. Null behaviour	11
Q2. Stability in n	11
Q3. Leakage into fitting items	13
Q4. Discrimination	15
Q5. Targeting	18
Run time	19
Interpretation	19
References	20

Purpose

In large samples, a calibrated item fit test detects misfit too small to matter. This study looks for an effect size that stays put as n grows. The candidate is a conditional RMSD (cRMSD), the root mean squared deviation between the observed and the model-implied conditional item characteristic curve, with the total score as the conditioning variable and CML item parameters. It is the numeric summary of what `RMitemICCPLOT()` draws. The literature RMSD is marginal (MML posterior pseudo-counts on a theta grid). A sum-score conditional version was not found. Background and the literature summary are in [conditional-rmsd-design.md](#).

Part A computes statistics only, with no bootstrap. Calibrated tests are Part B.

Definitions

For item i with maximum score m_i , let s be the total score, n_s the number of respondents with score s , N the number with a non-extreme total score, O_{is} the observed mean item score in group s , and E_{is} , V_{is} the conditional mean and variance under the Rasch model (`.cond_moments()`, from the elementary symmetric functions and the CML thresholds). With weights $w_s = n_s/N$ over non-extreme scores,

$$\text{MSD}_i = \frac{1}{m_i^2} \sum_s w_s (O_{is} - E_{is})^2, \quad \text{cRMSD}_i = \sqrt{\text{MSD}_i}.$$

Under the model, responses within a score group are independent, so $E[(O_{is} - E_{is})^2] = V_{is}/n_s$ and the bias of MSD_i is

$$B_i = \frac{1}{Nm_i^2} \sum_{s \in G} V_{is},$$

with G the observed non-extreme scores. The bias-corrected value is $\text{cRMSD}_i^{bc} = \sqrt{\max(\text{MSD}_i - B_i, 0)}$. Estimating the item location removes about one degree of freedom, so B_i should slightly overstate the null mean of MSD_i . The null condition checks this.

Two exact properties under CML, both checked on every dataset:

1. The signed mean deviation $\sum_s n_s (O_{is} - E_{is})$ is 0 (the CML score equation). A uniform shift is absorbed into the item location, so cRMSD measures the shape of the conditional curve only.
2. For dichotomous items, conditional infit is a linear contrast of the same residuals,

$$\text{Infit}_i - 1 = \frac{\sum_s n_s (O_{is} - E_{is})(1 - 2E_{is})}{\sum_s n_s V_{is}}.$$

Infit is a signed slope contrast, and cRMSD is the quadratic omnibus summary of the same residuals.

Two secondary variants are recorded:

- **Pooled.** Adjacent scores are merged until each group has at least 50 respondents. Pooling reduces noise but can hide deviations within a group.
- **Information-weighted.** Weights $w_s \propto n_s V_{is}$, which emphasise score groups where the item is informative, as infit does.

Predictions, stated before running

- **P1.** Under second-dimension misfit, infit discriminates misfitting from fitting items at least as well as cRMSD, and better at $\rho = .75$. Infit is the directed contrast for a flatter curve.
- **P2.** Bias-corrected cRMSD and infit MSQ are both stable in n . Raw cRMSD falls with n .
- **P3.** Leakage into fitting items is small relative to the misfitting items at $\rho = .50$, and closer at $\rho = .75$.
- **P4.** cRMSD of the misfitting item at -2 logits is smaller than at 0 logits, relative to infit.
- **P5.** Under asymmetric-IRF misfit (LPE), cRMSD discriminates better than infit, because an asymmetric deviation is roughly orthogonal to the slope contrast.
- **P6.** With three misfitting items at $\rho = .15$ and $.50$, where detection saturates, cRMSD and RMSEA separate misfitting from fitting items better than $|\text{infit} - 1|$ (lower false-positive rate at 95 percent detection, Q4), as S-X2 did in `sx2_comparison_v2.qmd`. The leakage into fitting items (Q3) is a smaller share of the misfitting items' value for cRMSD than for $|\text{infit} - 1|$.

Design

- **Items.** 20 dichotomous items with locations evenly spaced on $[-2, 2]$. The misfitting items are the ones nearest 0 , -1 and -2 logits. With one misfitting item it is the one nearest 0 .
- **Persons.** $\text{Theta} \sim N(0, 1.5^2)$.
- **Conditions.**
 - **null:** Rasch model, no misfit.
 - **xdim:** misfitting items respond to a second dimension, correlation $\rho = .50$ or $.75$ with the first, both SD 1.5 , with one or three misfitting items. Also $\rho = .15$ with three misfitting items, the strongest cascade in `sx2_comparison_v2.qmd`. The generator, the item locations and the choice of misfitting items are the same as in `sx2_comparison_v2.R`, so the cells at $n = 500$ to $2,000$ match that study.
 - **lpe:** misfitting items follow a logistic positive exponent IRF with a slope adjustment, $P(\theta) = \text{logistic}(a(\theta - b'))^\xi$, with b' chosen so that $P(b) = .5$. The curve is asymmetric but monotone. Without the adjustment ($a = 1$), the exponent also changes the effective slope (in a smoke test, $\xi = 4$ gave infit 0.85), so infit would detect it as slope misfit. a is chosen so that the item's mean infit is

1 at $n = 20,000$, separately for each misfit position. What remains is asymmetry with no net slope contrast, which is the case P5 is about. ξ and a are set in the calibration chunks below. One or three misfitting items.

- **Sample sizes.** 500, 1,000, 2,000, 5,000, 10,000, 20,000. The values at $n = 20,000$, averaged over replications, serve as the population values.
- **Replications.** 300 per cell, each a fresh draw (no subsampling of a master dataset).
- **Statistics per item.** Raw and bias-corrected cRMSD (primary: distribution weighted, unpooled), the pooled and information-weighted variants, conditional infit MSQ (`iarm::out_infit()`), the item-restscore gamma difference (observed minus expected, `iarm::item_restscore()`), and S-X2 with RMSEA (`mirt`). S-X2 uses `mirt`'s $df + 1$, the correction found in `sx2_null_diagnostic.qmd`, and RMSEA is recomputed from S-X2, the corrected df and N .

Relation to the S-X2 studies. `sx2_itemfit_sim.qmd` is superseded and is not used here. `sx2_comparison_v2.qmd` compared tests at a common family-wise error rate up to $n = 2,000$. Conditional infit with Westfall-Young detected more. The S-X2 bootstrap flagged far fewer fitting items when three items misfit at n of 1,000 or more, at nearly the same detection when misfit was strong or moderate ($\rho = .50$, $n = 1,000$: fitting items flagged in 3.5 against 39.5 percent of datasets, detection 93.8 against 98).

v2 attributes this to the bootstrap reference, because asymptotic S-X2 with BH cascaded more. A reanalysis of its cache (2026-10-05, p-values as stored) shows that the difference is BH, not the reference. BH controls the false discovery rate and loosens its threshold once the misfitting items are rejected. With Holm, which controls the family-wise rate like Westfall-Young, df -corrected asymptotic S-X2 flags fitting items about as rarely as the bootstrap, at the same detection:

rho	n	infit WY	infit Holm	S-X2 WY	boot	S-X2 Holm	df+1 BH	df+1
.15	1,000	97.0	90.5	6.5		11.0	34.0	
.15	2,000	100	100	47.5		55.0	93.0	
.50	1,000	39.5	28.5	3.5		4.5	14.5	
.50	2,000	87.0	72.5	7.5		9.0	28.0	

Datasets with at least one fitting item flagged (%), three misfitting items.

So the cascade resistance is a property of the S-X2 statistic. A likely mechanism follows from the infit identity above. The cascade puts small slope deviations on the fitting items. Infit is a one- df slope contrast and is sensitive to them. S-X2 spreads the same deviation over about 17 df and is not sensitive to it. Against the large deviation of a misfitting item both saturate. cRMSD is an omnibus summary like S-X2, so it may share this property (P6).

Part A asks a different question from v2: how the statistics behave as effect sizes, up to $n = 20,000$, with no test. The S-X2 columns here are therefore descriptive (RMSEA as an effect size, and the ROC on it), not a rerun of that comparison.

```
suppressMessages({
  devtools::load_all("../", quiet = TRUE) # for .cond_moments()
  library(psychotools)
  library(iarm)
  library(mirt)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
```

```

NCORES      <- 10
R_REPS      <- 300
NS          <- c(500, 1000, 2000, 5000, 10000, 20000)
K_ITEMS     <- 20
THETA_SD    <- 1.5
POOL_MIN    <- 50
INCLUDE_SX2 <- TRUE
CACHE_DIR   <- "conditional_rmsd"
dir.create(CACHE_DIR, showWarnings = FALSE)

locs <- seq(-2, 2, length.out = K_ITEMS)
misfit_idx <- integer(0)
for (tg in c(0, -1, -2)) {
  misfit_idx <- c(misfit_idx, setdiff(order(abs(locs - tg)), misfit_idx)[1])
}
data.frame(item = paste0("V", misfit_idx), target = c(0, -1, -2),
           location = round(locs[misfit_idx], 3))

```

item	target	location
V11	0	0.105
V6	-1	-0.947
V1	-2	-2.000

Data generation

```

# LPE item:  $P = \text{logistic}(a * (\text{theta} - b'))^{xi}$ , with  $b' = b - \text{shift} / a$  so
# that  $P(b) = .5$ .
lpe_shift <- function(xi) qlogis(0.5^(1 / xi))

# Slope that gives the LPE curve the Rasch slope (1/4) at  $P = .5$ . The
# derivative there is  $xi * a * 0.5 * (1 - 0.5^{(1/xi)})$ . Starting value for the
# numeric calibration below.
lpe_a_start <- function(xi) 0.5 / (xi * (1 - 0.5^(1 / xi)))

lpe_prob <- function(th, b, xi, a) {
  plogis(a * (th - (b - lpe_shift(xi) / a)))^xi
}

# `a_lpe` holds one slope per misfit position (0, -1, -2 logits). It is set
# by the calibration chunk and defaults to the analytic start.
a_lpe <- NULL

# `pos` selects which misfit positions are used (1 = 0 logits, 2 = -1,
# 3 = -2). The default is the first k. The calibration uses single positions.
gen_data <- function(type, k, par, n, a = a_lpe, pos = seq_len(k)) {
  use <- misfit_idx[pos]
  th <- rnorm(n, 0, THETA_SD)
  if (type == "xdim") {
    # Same generator as sx2_comparison_v2.R: fitting items respond to theta1,
    # misfitting items to theta2, both SD 1.5, correlation `par`.
    TH <- matrix(th, n, K_ITEMS)
    if (k > 0) {

```

```

    th2 <- par * th + sqrt(1 - par^2) * rnorm(n, 0, THETA_SD)
    TH[, use] <- th2
  }
  P <- plogis(TH - matrix(locs, n, K_ITEMS, byrow = TRUE))
  X <- (matrix(runif(n * K_ITEMS), n) < P) * 1L
} else {
  P <- plogis(outer(th, locs, "-"))
  if (type == "lpe" && k > 0) {
    if (is.null(a)) a <- rep(lpe_a_start(par), 3)
    for (p in pos) {
      j <- misfit_idx[p]
      P[, j] <- lpe_prob(th, locs[j], par, a[p])
    }
  }
  X <- (matrix(runif(n * K_ITEMS), n) < P) * 1L
}
X <- as.data.frame(X)
names(X) <- paste0("V", seq_len(K_ITEMS))
X
}

```

Statistics

```

# Merge adjacent score groups until each has at least `min_n` respondents.
pool_groups <- function(n_g, min_n) {
  id <- integer(length(n_g)); g <- 1L; acc <- 0
  for (j in seq_along(n_g)) {
    id[j] <- g; acc <- acc + n_g[j]
    if (acc >= min_n && j < length(n_g)) { g <- g + 1L; acc <- 0 }
  }
  if (acc < min_n && g > 1L) id[id == g] <- g - 1L # fold a short tail back
  id
}

compute_stats <- function(X) {
  m <- psychotools::pcmodel(X)
  thr <- lapply(psychotools::threshpar(m), as.numeric)
  mom <- .cond_moments(thr)
  m_i <- vapply(thr, length, integer(1))
  s <- rowSums(X)
  keep <- s > 0 & s < mom$max_score
  N <- sum(keep)
  ss <- s[keep]
  S_sum <- rowsum(as.matrix(X[keep, ]), ss) # groups x items
  g <- as.integer(rownames(S_sum))
  n_g <- as.numeric(table(ss)[as.character(g)])
  O <- S_sum / n_g
  E <- mom$E[g + 1, , drop = FALSE]
  V <- mom$V[g + 1, , drop = FALSE]
  D <- O - E

  # primary: distribution weighted, unpooled
  msd <- colSums(n_g * D^2) / N / m_i^2
  B <- colSums(V) / N / m_i^2

```

```

# information weighted
wv    <- n_g * V
msd_w <- colSums(wv * D^2) / colSums(wv) / m_i^2
B_w   <- colSums(V^2) / colSums(wv) / m_i^2
# pooled
pid   <- pool_groups(n_g, POOL_MIN)
n_p   <- as.numeric(tapply(n_g, pid, sum))
O_p   <- rowsum(S_sum, pid) / n_p
E_p   <- rowsum(n_g * E, pid) / n_p
B_p   <- colSums(rowsum(n_g * V, pid) / n_p) / N / m_i^2
msd_p <- colSums(n_p * (O_p - E_p)^2) / N / m_i^2

infit <- iarm::out_infit(m)$Infit
infit_lin <- 1 + colSums(n_g * D * (1 - 2 * E)) / colSums(n_g * V)

rs <- iarm::item_restscore(m)
gdifff <- as.numeric(rs[, "observed"]) - as.numeric(rs[, "expected"])

out <- data.frame(
  item = seq_len(ncol(X)),
  msd = msd, B = B,
  crmsd_raw = sqrt(msd),
  crmsd_bc = sqrt(pmax(msd - B, 0)),
  crmsd_w_bc = sqrt(pmax(msd_w - B_w, 0)),
  crmsd_p_bc = sqrt(pmax(msd_p - B_p, 0)),
  infit = infit,
  gamma_diff = gdifff,
  md = colSums(n_g * D) / N,
  infit_identity_err = abs(infit - infit_lin),
  n_groups = length(g), n_pooled = length(n_p)
)

if (INCLUDE_SX2) {
  sx <- tryCatch({
    mm <- mirt::mirt(X, 1, itemtype = "Rasch", verbose = FALSE)
    f <- mirt::itemfit(mm, fit_stats = "S_X2")
    # mirt removes one df per item for the estimated latent variance,
    # which is one too many for the Rasch model (sx2_null_diagnostic.qmd).
    # Use df + 1 for both the p-value and RMSEA.
    data.frame(sx2 = f$S_X2, sx2_df = f$df.S_X2 + 1)
  }, error = function(e) data.frame(sx2 = NA, sx2_df = NA))
  out <- cbind(out, sx[seq_len(ncol(X)), ])
  out$sx2_p <- pchisq(out$sx2, out$sx2_df, lower.tail = FALSE)
  out$sx2_rmsea <- sqrt(pmax(0, (out$sx2 - out$sx2_df) /
    (out$sx2_df * (nrow(X) - 1))))
}
out
}

```

Calibrating the LPE item

The LPE condition is meant as misfit of moderate size with no net slope contrast, so that infit is near 1 while the conditional curve is still wrong. Two steps.

Step 1: choose ξ . For several values of ξ , with the analytic slope `lpe_a_start(xi)` (the Rasch slope at $P = .5$), the table gives the bias-corrected cRMSD and the infit of a single LPE item at 0 logits, at $n = 20,000$ (5 datasets each). Choose ξ so that cRMSD is roughly .03 to .05 (small to medium on the Köhler et al., 2020, scale). Record the choice and the reason here **before** the main run.

```
cal_file <- file.path(CACHE_DIR, "lpe_xi_table.rds")
if (file.exists(cal_file)) {
  cal <- readRDS(cal_file)
} else {
  cal <- do.call(rbind, lapply(c(0.1, 0.25, 0.5, 2, 4, 8, 16), function(xi) {
    a0 <- lpe_a_start(xi)
    do.call(rbind, lapply(1:5, function(r) {
      set.seed(9e5 + 100 * xi + r)
      X <- gen_data("lpe", 1, xi, 20000, a = rep(a0, 3), pos = 1)
      st <- compute_stats(X)
      data.frame(xi = xi, a_start = a0, rep = r,
                  st[misfit_idx[1], c("crmsd_bc", "infit", "gamma_diff")])
    })))
  saveRDS(cal, cal_file)
}
cal |>
  group_by(xi) |>
  summarise(across(c(a_start, crmsd_bc, infit, gamma_diff),
                    \ (x) round(mean(x), 3)))
```

xi	a_start	crmsd_bc	infit	gamma_diff
0.10	5.005	0.039	0.967	0.028
0.25	2.133	0.026	0.996	0.004
0.50	1.333	0.013	1.009	-0.007
2.00	0.854	0.006	0.992	0.006
4.00	0.786	0.013	0.986	0.012
8.00	0.753	0.016	0.978	0.018
16.00	0.737	0.020	0.974	0.021

```
XI <- 0.1
```

Choice, recorded 2026-10-05 before the main run. With the analytic slope, only $\xi = 0.1$ reached the target range (cRMSD .039, infit 0.97). The other values gave .006 to .026. The slope adjustment removes most of what an exponent does to the curve, so a strongly asymmetric curve is needed for a cRMSD in the small-to-medium range. $\xi = 0.1$ makes the curve rise gradually at low theta and steeply near and above its location.

Step 2: set the slope for each position. For the chosen ξ , find the a that makes the item's mean infit equal 1 at $n = 20,000$, separately for the items at 0, -1 and -2 logits, each as the only LPE item. The same three seeds are used at every candidate a (common random numbers), so mean infit is a smooth function of a and `uniroot()` can solve for it. The check table then reports the item's infit and cRMSD at the calibrated slope on five new datasets. In the three-item condition the other LPE items shift the total score slightly, so infit there will be close to 1 but not exactly 1. Changing ξ or these slopes later means deleting the `lpe_*` cache files.

```

slope_file <- file.path(CACHE_DIR, sprintf("lpe_slopes_xi%s.rds", XI))
if (file.exists(slope_file)) {
  slope_cal <- readRDS(slope_file)
} else {
  slope_cal <- do.call(rbind, lapply(1:3, function(p) {
    j <- misfit_idx[p]
    mean_infit <- function(a) {
      aa <- rep(lpe_a_start(XI), 3); aa[p] <- a
      mean(vapply(1:3, function(r) {
        set.seed(8e5 + 10 * p + r)
        compute_stats(gen_data("lpe", 1, XI, 20000, a = aa, pos = p))$infit[j]
      }, numeric(1)))
    }
    a0 <- lpe_a_start(XI)
    root <- uniroot(function(a) mean_infit(a) - 1,
                    interval = c(a0 / 2, a0 * 2), tol = 1e-3)
    data.frame(position = p, target = c(0, -1, -2)[p], a_start = a0,
              a = root$root, iterations = root$iter)
  )))
  saveRDS(slope_cal, slope_file)
}
a_lpe <- slope_cal$a

check_file <- file.path(CACHE_DIR, sprintf("lpe_slope_check_xi%s.rds", XI))
if (file.exists(check_file)) {
  slope_check <- readRDS(check_file)
} else {
  slope_check <- do.call(rbind, lapply(1:3, function(p) {
    do.call(rbind, lapply(1:5, function(r) {
      set.seed(7e5 + 10 * p + r)
      st <- compute_stats(gen_data("lpe", 1, XI, 20000, pos = p))
      data.frame(position = p, st[misfit_idx[p], c("crmsd_bc", "infit",
                                                    "gamma_diff")])
    })))
  saveRDS(slope_check, check_file)
}
slope_cal |> mutate(across(c(a_start, a), \(x) round(x, 3)))

```

position	target	a_start	a	iterations
1	0	5.005	4.589	7
2	-1	5.005	3.584	8
3	-2	5.005	2.826	7

```

slope_check |>
  group_by(position) |>
  summarise(across(everything(), \(x) round(mean(x), 3)))

```

position	crmsd_bc	infit	gamma_diff
1	0.036	0.994	0.008

position	crmsd_bc	infit	gamma_diff
2	0.037	1.011	0.015
3	0.027	1.008	0.029

If `uniroot()` fails because no slope in $[a/2, 2a]$ gives infit 1, choose another XI.

Run

One cache file per condition and sample size. Delete a file to rerun that cell.

```
conds <- tribble(
  ~cond,      ~type, ~k, ~par,
  "null",     "null", 0, NA,
  "xdim15_k3", "xdim", 3, 0.15,
  "xdim50_k1", "xdim", 1, 0.50,
  "xdim50_k3", "xdim", 3, 0.50,
  "xdim75_k1", "xdim", 1, 0.75,
  "xdim75_k3", "xdim", 3, 0.75,
  "lpe_k1",    "lpe",  1, XI,
  "lpe_k3",    "lpe",  3, XI
)
conds
```

cond	type	k	par
null	null	0	NA
xdim15_k3	xdim	3	0.15
xdim50_k1	xdim	1	0.50
xdim50_k3	xdim	3	0.50
xdim75_k1	xdim	1	0.75
xdim75_k3	xdim	3	0.75
lpe_k1	lpe	1	0.10
lpe_k3	lpe	3	0.10

```
run_cell <- function(ci, n) {
  cc <- conds[ci, ]
  f <- file.path(CACHE_DIR, sprintf("%s_n%d.rds", cc$cond, n))
  if (file.exists(f)) return(readRDS(f))
  t0 <- Sys.time()
  res <- mclapply(seq_len(R_REPS), function(r) {
    set.seed(1e7 * ci + 100 * (n %% 100) + r)
    out <- tryCatch({
      X <- gen_data(cc$type, cc$k, cc$par, n)
      st <- compute_stats(X)
      st$rep <- r
      st
    }, error = function(e) conditionMessage(e))
    out
  }, mc.cores = NCORES)
  ok <- vapply(res, is.data.frame, logical(1))
  df <- do.call(rbind, res[ok])
}
```

```

df$cond <- cc$cond; df$type <- cc$type; df$k <- cc$k; df$par <- cc$par
df$n <- n
attr(df, "failed") <- unlist(res[!ok])
attr(df, "minutes") <- as.numeric(Sys.time() - t0, units = "mins")
saveRDS(df, f)
df
}

cells <- expand.grid(ci = seq_len(nrow(conds)), n = NS)
all_res <- lapply(seq_len(nrow(cells)), function(j) run_cell(cells$ci[j],
                                                              cells$n[j]))

timing <- data.frame(
  cond = conds$cond[cells$ci], n = cells$n,
  failed = vapply(all_res, function(d) length(attr(d, "failed")), integer(1)),
  minutes = round(vapply(all_res, function(d) attr(d, "minutes") %||% NA_real_,
                        numeric(1)), 2)
)
res <- bind_rows(all_res) |>
  mutate(misfit = mapply(function(k, it) it %in% misfit_idx[seq_len(k)],
                        k, item),
         target = c("0", "-1", "-2")[match(item, misfit_idx)],
         abs_infit = abs(infit - 1),
         abs_gamma = abs(gamma_diff),
         # Untruncated scores for the ROC. crmsd_bc and RMSEA are 0 for most
         # null items, and the ties make thresholds at 0 flag every item.
         # Both scores rank items like the truncated values where those are
         # positive.
         crmsd_score = msd - B,
         sx2_score = if (INCLUDE_SX2) (sx2 - sx2_df) / sx2_df else NA,
         n = factor(n, levels = NS))

```

Checks

Failed datasets, the CML score equation (signed mean deviation should be 0 up to the CML convergence tolerance), and the infit identity (difference from `iarm::out_infit()`, which returns infit unrounded).

```
timing |> filter(failed > 0)
```

```
cond  n  failed  minutes
```

```

res |>
  group_by(cond) |>
  summarise(max_abs_md = signif(max(abs(md)), 2),
            max_infit_identity_err = signif(max(infit_identity_err), 2),
            datasets = n_distinct(paste(n, rep)))

```

cond	max_abs_md	max_infit_identity_err	datasets
lpe_k1	9.3e-05	0	1800
lpe_k3	9.8e-05	0	1800
null	2.0e-04	0	1800
xdim15_k3	8.1e-05	0	1800

cond	max_abs_md	max_infit_identity_err	datasets
xdim50_k1	2.0e-04	0	1800
xdim50_k3	7.8e-05	0	1800
xdim75_k1	1.4e-04	0	1800
xdim75_k3	2.0e-04	0	1800

Q1. Null behaviour

Raw and bias-corrected cRMSD under a true Rasch model, and the ratio of the mean MSD to the mean analytic bias B_i . A ratio below 1 is the degree of freedom taken by the item location.

```
res |>
  filter(cond == "null") |>
  group_by(n) |>
  summarise(raw_mean = mean(crmsd_raw), bc_mean = mean(crmsd_bc),
            bc_p95 = quantile(crmsd_bc, .95), bc_zero = mean(crmsd_bc == 0),
            msd_over_B = mean(msd) / mean(B),
            pooled_bc_mean = mean(crmsd_p_bc), info_bc_mean = mean(crmsd_w_bc),
            infit_sd = sd(infit), .groups = "drop") |>
  mutate(across(where(is.numeric), \(x) round(x, 4)))
```

n	raw_mean	bc_mean	bc_p95	bc_zero	msd_over_B	pooled_bc_mean	info_bc_mean	infit_sd
500	0.0682	0.0128	0.0548	0.6438	0.9201	0.0082	0.0152	0.0509
1000	0.0484	0.0093	0.0391	0.6388	0.9281	0.0087	0.0109	0.0361
2000	0.0343	0.0065	0.0276	0.6352	0.9269	0.0065	0.0076	0.0252
5000	0.0216	0.0041	0.0177	0.6407	0.9232	0.0041	0.0048	0.0161
10000	0.0153	0.0029	0.0125	0.6432	0.9198	0.0029	0.0034	0.0114
20000	0.0108	0.0020	0.0088	0.6448	0.9161	0.0020	0.0024	0.0080

Q2. Stability in n

Mean values for the misfitting items by sample size. A statistic is stable if its mean does not drift with n. The n = 20,000 row is the population proxy.

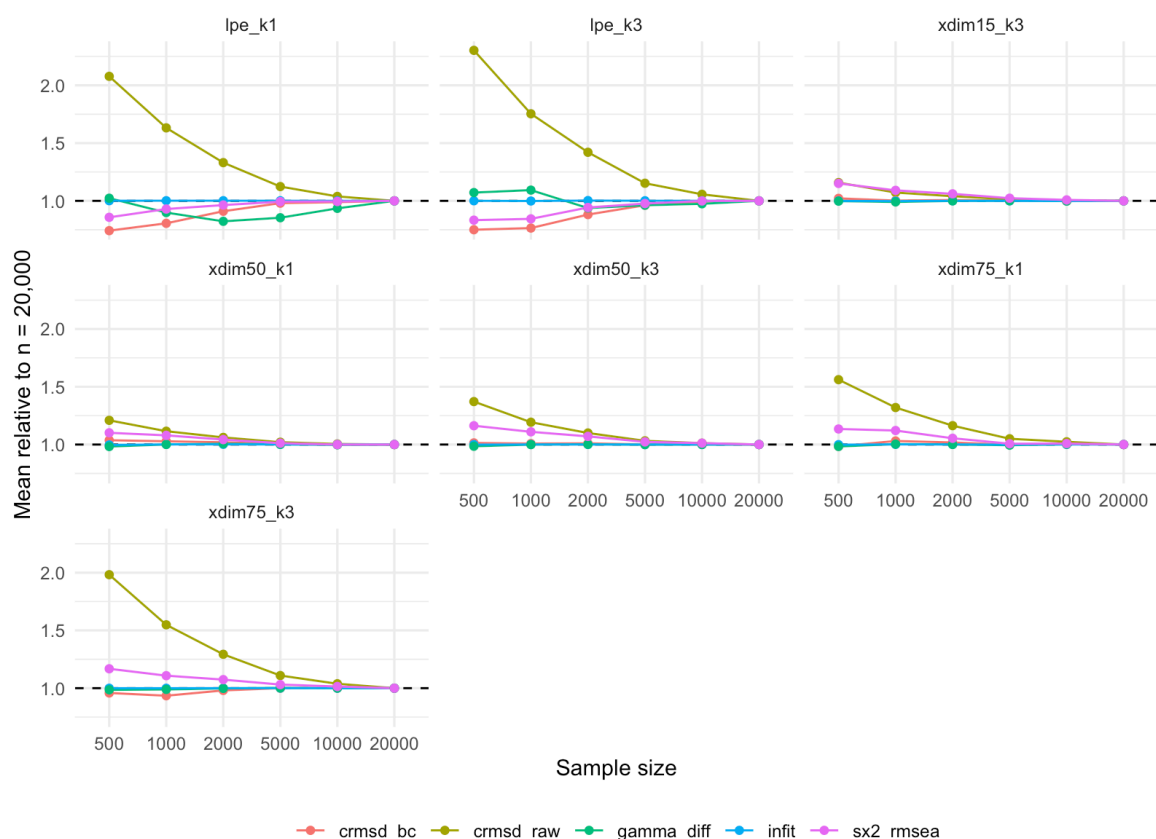
```
q2 <- res |>
  filter(misfit) |>
  group_by(cond, n) |>
  summarise(crmsd_raw = mean(crmsd_raw), crmsd_bc = mean(crmsd_bc),
            crmsd_p_bc = mean(crmsd_p_bc), infit = mean(infit),
            gamma_diff = mean(gamma_diff),
            sx2_rmsea = if (INCLUDE_SX2) mean(sx2_rmsea) else NA,
            .groups = "drop")
q2 |> mutate(across(where(is.numeric), \(x) round(x, 3)))
```

cond	n	crmsd_raw	crmsd_bc	crmsd_p_bc	infit	gamma_diff	sx2_rmsea
lpe_k1	500	0.081	0.027	0.025	0.999	0.005	0.022
lpe_k1	1000	0.063	0.030	0.030	1.000	0.004	0.023
lpe_k1	2000	0.052	0.034	0.033	1.000	0.004	0.024

cond	n	crmsd_raw	crmsd_bc	crmsd_p_bc	infit	gamma_diff	sx2_rmsea
lpe_k1	5000	0.044	0.036	0.036	0.999	0.004	0.025
lpe_k1	10000	0.040	0.036	0.036	0.999	0.004	0.025
lpe_k1	20000	0.039	0.037	0.037	0.998	0.005	0.025
lpe_k3	500	0.074	0.023	0.019	0.999	0.021	0.018
lpe_k3	1000	0.057	0.023	0.023	0.997	0.021	0.018
lpe_k3	2000	0.046	0.027	0.026	1.001	0.018	0.020
lpe_k3	5000	0.037	0.029	0.029	0.999	0.019	0.021
lpe_k3	10000	0.034	0.030	0.030	0.999	0.019	0.021
lpe_k3	20000	0.032	0.030	0.030	0.998	0.020	0.021
xdim15_k3	500	0.156	0.137	0.129	1.359	-0.369	0.108
xdim15_k3	1000	0.144	0.134	0.132	1.356	-0.366	0.102
xdim15_k3	2000	0.140	0.135	0.134	1.360	-0.370	0.099
xdim15_k3	5000	0.136	0.134	0.134	1.359	-0.370	0.096
xdim15_k3	10000	0.135	0.134	0.134	1.358	-0.369	0.094
xdim15_k3	20000	0.134	0.134	0.134	1.359	-0.370	0.093
xdim50_k1	500	0.148	0.126	0.119	1.372	-0.286	0.097
xdim50_k1	1000	0.136	0.125	0.123	1.378	-0.291	0.095
xdim50_k1	2000	0.130	0.124	0.124	1.379	-0.293	0.092
xdim50_k1	5000	0.125	0.122	0.122	1.378	-0.291	0.089
xdim50_k1	10000	0.123	0.122	0.122	1.376	-0.290	0.088
xdim50_k1	20000	0.122	0.122	0.122	1.377	-0.291	0.088
xdim50_k3	500	0.110	0.081	0.074	1.225	-0.206	0.063
xdim50_k3	1000	0.096	0.080	0.078	1.226	-0.210	0.060
xdim50_k3	2000	0.089	0.080	0.080	1.226	-0.210	0.058
xdim50_k3	5000	0.083	0.080	0.080	1.225	-0.209	0.056
xdim50_k3	10000	0.081	0.080	0.080	1.225	-0.210	0.055
xdim50_k3	20000	0.081	0.080	0.080	1.226	-0.210	0.054
xdim75_k1	500	0.101	0.063	0.058	1.194	-0.145	0.050
xdim75_k1	1000	0.086	0.066	0.063	1.197	-0.148	0.049
xdim75_k1	2000	0.076	0.065	0.065	1.196	-0.148	0.046
xdim75_k1	5000	0.068	0.064	0.064	1.194	-0.147	0.044
xdim75_k1	10000	0.066	0.064	0.064	1.196	-0.148	0.044
xdim75_k1	20000	0.065	0.064	0.064	1.195	-0.148	0.044
xdim75_k3	500	0.084	0.039	0.033	1.119	-0.101	0.032
xdim75_k3	1000	0.066	0.038	0.036	1.119	-0.101	0.030
xdim75_k3	2000	0.055	0.040	0.040	1.119	-0.102	0.029
xdim75_k3	5000	0.047	0.041	0.041	1.120	-0.102	0.028

cond	n	crmsd_raw	crmsd_bc	crmsd_p_bc	infit	gamma_diff	sx2_rmsea
xdim75_k3	10000	0.044	0.041	0.041	1.119	-0.102	0.028
xdim75_k3	20000	0.042	0.041	0.041	1.119	-0.102	0.027

```
q2 |>
  pivot_longer(c(crmsd_raw, crmsd_bc, infit, gamma_diff, sx2_rmsea),
               names_to = "stat") |>
  group_by(cond, stat) |>
  mutate(rel = value / value[n == "20000"]) |>
  ggplot(aes(n, rel, colour = stat, group = stat)) +
  geom_hline(yintercept = 1, linetype = 2) +
  geom_line() + geom_point() +
  facet_wrap(~cond) +
  labs(x = "Sample size", y = "Mean relative to n = 20,000", colour = NULL) +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom")
```



Q3. Leakage into fitting items

Mean values for the *fitting* items, by condition, at each n , against the null. The last table compares fitting and misfitting items at $n = 20,000$, using each item's mean over replications as its population value. It gives the mean over items and the most extreme item in each group: the largest fitting item and the smallest misfitting item. If the smallest misfitting item stands well above the largest fitting item, an effect-size floor between the two can stop the cascade at any n .

```
res |>
  filter(!misfit) |>
```

```

group_by(cond, n) |>
summarise(crmsd_bc = mean(crmsd_bc), infit_signed = mean(infit - 1),
          abs_infit = mean(abs_infit), abs_gamma = mean(abs_gamma),
          .groups = "drop") |>
filter(n %in% c("500", "2000", "20000")) |>
mutate(across(where(is.numeric), \(x) round(x, 4)))

```

cond	n	crmsd_bc	infit_signed	abs_infit	abs_gamma
lpe_k1	500	0.0131	0.0000	0.0412	0.0321
lpe_k1	2000	0.0068	0.0002	0.0206	0.0157
lpe_k1	20000	0.0024	0.0003	0.0072	0.0054
lpe_k3	500	0.0134	0.0032	0.0410	0.0309
lpe_k3	2000	0.0071	0.0029	0.0218	0.0164
lpe_k3	20000	0.0039	0.0033	0.0104	0.0070
null	500	0.0128	0.0001	0.0406	0.0315
null	2000	0.0065	-0.0001	0.0201	0.0156
null	20000	0.0020	0.0000	0.0064	0.0050
xdim15_k3	500	0.0149	-0.0677	0.0701	0.0698
xdim15_k3	2000	0.0190	-0.0679	0.0679	0.0669
xdim15_k3	20000	0.0257	-0.0678	0.0678	0.0664
xdim50_k1	500	0.0124	-0.0221	0.0432	0.0362
xdim50_k1	2000	0.0071	-0.0225	0.0272	0.0226
xdim50_k1	20000	0.0057	-0.0223	0.0223	0.0185
xdim50_k3	500	0.0129	-0.0415	0.0515	0.0464
xdim50_k3	2000	0.0100	-0.0418	0.0427	0.0385
xdim50_k3	20000	0.0141	-0.0415	0.0415	0.0374
xdim75_k1	500	0.0132	-0.0112	0.0408	0.0329
xdim75_k1	2000	0.0063	-0.0116	0.0219	0.0177
xdim75_k1	20000	0.0029	-0.0113	0.0118	0.0097
xdim75_k3	500	0.0128	-0.0217	0.0431	0.0361
xdim75_k3	2000	0.0072	-0.0214	0.0270	0.0226
xdim75_k3	20000	0.0058	-0.0214	0.0215	0.0182

```

res |>
filter(n == "20000", cond != "null") |>
group_by(cond, misfit, item) |>
summarise(crmsd_bc = mean(crmsd_bc), abs_infit = mean(abs_infit),
          abs_gamma = mean(abs_gamma), .groups = "drop") |>
group_by(cond, misfit) |>
summarise(crmsd_mean = mean(crmsd_bc),
          crmsd_extreme = if (first(misfit)) min(crmsd_bc) else max(crmsd_bc),
          abs_infit_mean = mean(abs_infit),
          abs_infit_extreme = if (first(misfit)) min(abs_infit) else max(abs_infit),

```

```

abs_gamma_mean = mean(abs_gamma),
abs_gamma_extreme = if (first(misfit)) min(abs_gamma) else max(abs_gamma),
.groups = "drop") |>
mutate(across(where(is.numeric), \(x) round(x, 4)))

```

cond	misfit	crmsd_mean	crmsd_extreme	abs_infit_mean	abs_infit_extreme	abs_gamma_mean	abs_gamma_extreme
lpe_k1	FALSE	0.0024	0.0027	0.0072	0.0083	0.0054	0.0064
lpe_k1	TRUE	0.0369	0.0369	0.0066	0.0066	0.0059	0.0059
lpe_k3	FALSE	0.0039	0.0052	0.0104	0.0148	0.0070	0.0096
lpe_k3	TRUE	0.0302	0.0241	0.0066	0.0059	0.0204	0.0046
xdim15_k	FALSE	0.0257	0.0318	0.0678	0.0783	0.0664	0.0822
xdim15_k	TRUE	0.1339	0.1005	0.3593	0.2719	0.3700	0.3552
xdim50_k	FALSE	0.0057	0.0064	0.0223	0.0238	0.0185	0.0197
xdim50_k	TRUE	0.1218	0.1218	0.3765	0.3765	0.2906	0.2906
xdim50_k	FALSE	0.0141	0.0178	0.0415	0.0497	0.0374	0.0470
xdim50_k	TRUE	0.0797	0.0595	0.2256	0.1735	0.2097	0.1950
xdim75_k	FALSE	0.0029	0.0033	0.0118	0.0134	0.0097	0.0105
xdim75_k	TRUE	0.0638	0.0638	0.1955	0.1955	0.1478	0.1478
xdim75_k	FALSE	0.0058	0.0079	0.0215	0.0270	0.0182	0.0233
xdim75_k	TRUE	0.0407	0.0302	0.1191	0.0927	0.1021	0.0917

Q4. Discrimination

Misfitting against fitting items in the same datasets, for each statistic. All statistics are made direction-free: cRMSD, $|\text{infit} - 1|$, $|\text{gamma difference}|$, and S-X2. cRMSD enters as MSD - B and S-X2 as $(\text{S-X2} - \text{df})/\text{df}$, the untruncated quantities behind bias-corrected cRMSD and RMSEA. They rank items like the truncated values where those are positive, without the ties at 0. The pooled and information-weighted cRMSD variants are left out of the ROC for the same reason, since only their truncated values were stored. For each cell: the AUC, the false-positive rate on fitting items at a matched detection rate of the misfitting items (50, 80 and 95 percent), and the partial AUC up to a false-positive rate of .05, normalised to $[0, 1]$. Items within a dataset are not independent, so treat small differences with caution.

```

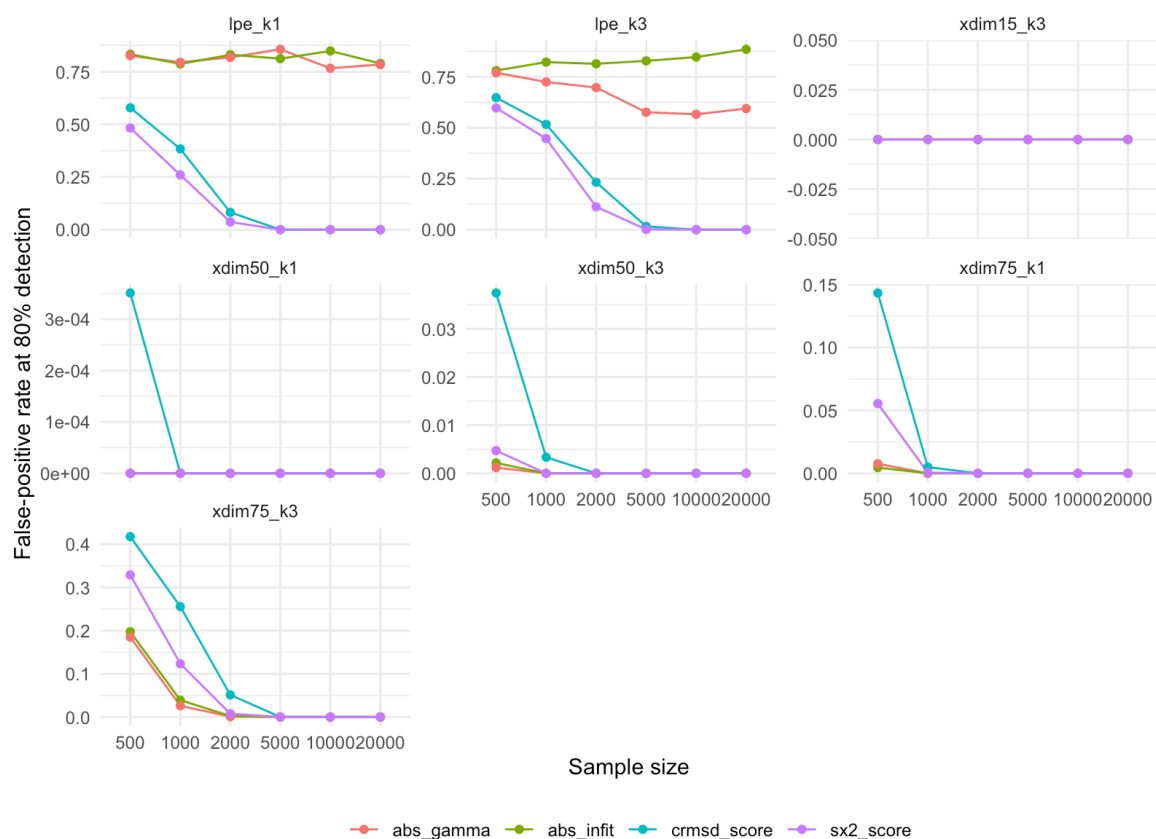
roc_summ <- function(pos, neg) {
  auc <- mean(outer(pos, neg, ">") + 0.5 * outer(pos, neg, "=="))
  fpr_at <- function(tpr) mean(neg >= quantile(pos, 1 - tpr, type = 1))
  thr <- sort(unique(c(pos, neg)), decreasing = TRUE)
  fpr <- vapply(thr, function(t) mean(neg >= t), numeric(1))
  tpr <- vapply(thr, function(t) mean(pos >= t), numeric(1))
  sel <- fpr <= 0.05
  pauc <- if (sum(sel) > 1) sum(diff(fpr[sel]) * head(tpr[sel], -1)) / 0.05 else 0
  data.frame(auc = auc, fpr_tpr50 = fpr_at(.5), fpr_tpr80 = fpr_at(.8),
    fpr_tpr95 = fpr_at(.95), pauc05 = pauc)
}
stats_q4 <- c("crrmsd_score", "abs_infit", "abs_gamma",
  if (INCLUDE_SX2) "sx2_score")

```

```
q4 <- res |>
  filter(cond != "null") |>
  group_by(cond, n) |>
  group_modify(function(d, key) {
    bind_rows(lapply(stats_q4, function(st) {
      cbind(stat = st, roc_summ(d[[st]][d$misfit], d[[st]][!d$misfit]))
    })))
  }) |>
  ungroup()

q4 |>
  select(cond, n, stat, auc, fpr_tpr80, fpr_tpr95, pauc05) |>
  mutate(across(where(is.numeric), \(x) round(x, 3))) |>
  pivot_wider(names_from = stat, values_from = c(auc, fpr_tpr80, fpr_tpr95, pauc05))
```

[illegible]



Q5. Targeting

Values for each misfitting item at 0, -1 and -2 logits, in the three-item conditions, at $n = 2,000$ and $n = 20,000$.

```
res |>
  filter(misfit, k == 3, n %in% c("2000", "20000")) |>
  group_by(cond, n, target) |>
  summarise(crmsd_bc = mean(crmsd_bc), infit = mean(infit),
            gamma_diff = mean(gamma_diff),
            sx2_rmsea = if (INCLUDE_SX2) mean(sx2_rmsea) else NA,
            .groups = "drop") |>
  arrange(cond, n, factor(target, levels = c("0", "-1", "-2"))) |>
  mutate(across(where(is.numeric), \(x) round(x, 3)))
```

cond	n	target	crmsd_bc	infit	gamma_diff	sx2_rmsea
lpe_k3	2000	0	0.031	1.001	0.002	0.022
lpe_k3	2000	-1	0.028	1.001	0.020	0.021
lpe_k3	2000	-2	0.021	1.001	0.034	0.018
lpe_k3	20000	0	0.035	1.000	0.002	0.023
lpe_k3	20000	-1	0.032	0.999	0.021	0.022
lpe_k3	20000	-2	0.024	0.996	0.036	0.019
xdim15_k3	2000	0	0.161	1.432	-0.384	0.112
xdim15_k3	2000	-1	0.142	1.375	-0.371	0.101
xdim15_k3	2000	-2	0.101	1.273	-0.354	0.084

cond	n	target	crmsd_bc	infit	gamma_diff	sx2_rmsea
xdim15_k3	20000	0	0.160	1.432	-0.384	0.106
xdim15_k3	20000	-1	0.141	1.374	-0.371	0.097
xdim15_k3	20000	-2	0.101	1.272	-0.355	0.077
xdim50_k3	2000	0	0.098	1.271	-0.224	0.067
xdim50_k3	2000	-1	0.085	1.233	-0.212	0.060
xdim50_k3	2000	-2	0.059	1.174	-0.195	0.047
xdim50_k3	20000	0	0.096	1.270	-0.222	0.063
xdim50_k3	20000	-1	0.084	1.234	-0.212	0.056
xdim50_k3	20000	-2	0.059	1.174	-0.195	0.043
xdim75_k3	2000	0	0.050	1.144	-0.111	0.034
xdim75_k3	2000	-1	0.041	1.122	-0.103	0.030
xdim75_k3	2000	-2	0.029	1.093	-0.091	0.023
xdim75_k3	20000	0	0.050	1.143	-0.111	0.032
xdim75_k3	20000	-1	0.042	1.122	-0.103	0.028
xdim75_k3	20000	-2	0.030	1.093	-0.092	0.021

Run time

```
timing |> group_by(n) |> summarise(minutes = sum(minutes, na.rm = TRUE))
```

n	minutes
500	0.68
1000	0.76
2000	0.88
5000	1.45
10000	2.28
20000	3.81

Interpretation

Written 2026-10-05 after the run. 14,400 datasets, none failed, 9.9 minutes on 10 cores. The CML mean deviation was at most $2e-4$ and the infit identity held to $3e-13$ in every dataset.

1. The bias correction works, with the expected limits. Under the null, raw cRMSD falls from .068 at $n = 500$ to .011 at $n = 20,000$. The bias-corrected value is 0 for about 64 percent of items at every n , and its mean (.013 to .002) is what truncation at zero leaves. The mean MSD is 0.92 of the analytic bias B_i at every n , the one degree of freedom taken by the item location. Under misfit, bias-corrected cRMSD is within .003 of its $n = 20,000$ value from $n = 1,000$ when the population value is .06 or more. For smaller values it is biased downward at small n (.027 at $n = 500$ against .037 for the single LPE item), as Robitzsch (2022) found for the marginal RMSD.

2. Infit MSQ and the gamma difference are stable effect sizes at every n (P2 holds for infit, and is wrong about the gamma difference). Their means for the misfitting items move by at most .01 between $n = 500$ and $n = 20,000$ in any condition. Both are differences or linear contrasts, not squares, so neither carries the positive bias of an RMSD. RMSEA drifts down by 9 to 16 percent over the same range.

3. Under second-dimension misfit, cRMSD and infit carry the same information. For every item whose infit departs from 1 by more than .05, cRMSD is 0.34 to 0.41 times $|\text{infit} - 1|$. Their leakage shares are about the same. The share of the misfitting items' value that shows up on fitting items is 18 to 19 percent for both with three misfitting items at $\rho = .15$ and $.50$, and 5 to 6 percent with one misfitting item. Their fall-off with targeting is also the same: the item at -2 logits keeps about 0.62 of the value of the item at 0 on both scales (P4 does not hold). The gamma difference keeps 0.83 to 0.92, so it is the least sensitive to targeting. RMSEA keeps 0.66 to 0.73.

4. Infit and the gamma difference rank items better than cRMSD at small n (P1 holds). Where the ranking is not perfect ($\rho = .75$ with three misfitting items, n up to 2,000, and $\rho = .75$ with one at $n = 500$), $|\text{infit} - 1|$ and $|\text{gamma difference}|$ have the highest AUC and partial AUC, S-X2 is next and cRMSD last. At $n = 2,000$ and $\rho = .75$ with three misfitting items, the false-positive rate at 95 percent detection is .047 for infit, .025 for the gamma difference, .161 for S-X2 and .301 for cRMSD.

5. As effect sizes, all four separate misfitting items from cascade-affected fitting items (P6 does not hold). In the saturated cells ($\rho = .15$ and $.50$) every statistic reaches AUC 1. At $n = 20,000$ the smallest misfitting item exceeds the largest fitting item by a factor of 3.1 to 3.8 for cRMSD, 3.4 to 3.5 for $|\text{infit} - 1|$ and 3.9 to 4.3 for the gamma difference, with three misfitting items. The leakage is real and does not shrink with n . At $\rho = .15$, fitting items have mean infit 0.93 at every n , which is why an infit *test* flags them as n grows. But it stays well below the misfitting items. So S-X2's resistance to the cascade in `sx2_comparison_v2.qmd` belongs to testing, not to ranking. An omnibus test spends its power over many df and does not reach significance for the small slope deviations of the fitting items. As an effect size, infit ranks the items as well as S-X2 does.

6. cRMSD earns its place on misfit that is not a slope (P5 holds). With the LPE items calibrated to infit 1, $|\text{infit} - 1|$ has an AUC of .34 to .51 at every n , which is no better than chance. cRMSD and S-X2 reach AUC .94 to .97 at $n = 2,000$ with one LPE item and 1.00 from $n = 5,000$. S-X2 is a little better than cRMSD at moderate n . The gamma difference picks up part of it with three LPE items (AUC .62 to .77). That misfit is small, though: cRMSD .024 to .037. A monotone curve whose slope matches the Rasch curve can only depart from it by a little, so misfit that infit cannot see is likely to be small too.

What this means for large samples. The large-sample problem is a testing problem. The effect sizes are stable, and under second-dimension misfit they put the misfitting items well above the cascade-affected ones at every n . An effect-size floor would therefore stop the cascade. The floor can sit on infit MSQ, which needs no bias correction and ranks best, or on the gamma difference, which is the least sensitive to targeting. Within a condition the gap is wide. Across conditions it is narrower: the smallest misfitting item at $\rho = .75$ ($|\text{infit} - 1| = .093$) is close to the largest fitting item at $\rho = .15$ (.078). So one fixed floor for every scale would not work, and the floor should come from impact (option D in `HANDOFF-large-sample-fit.md`). cRMSD adds probability units and sensitivity to non-slope misfit. It does not replace infit.

Limits. Dichotomous items only, 20 items, normal theta, and two misfit mechanisms. The LPE condition was calibrated to infit 1 by design, so P5 tests detection of non-slope misfit when it exists, not how often it occurs. Items within a dataset are not independent, so small ROC differences are uncertain. The pooled and information-weighted cRMSD variants were stored only in truncated form and are not in the ROC.

References

- Köhler, C., Robitzsch, A., & Hartig, J. (2020). A bias-corrected RMSD item fit statistic: An evaluation and comparison to alternatives. *Journal of Educational and Behavioral Statistics*, 45(3), 251-273.
- Robitzsch, A. (2022). Statistical properties of estimators of the RMSD item fit statistic. *Foundations*, 2, 488-503.